



MVSR ENGINEERING COLLEGE

UGC AUTONOMOUS

Approved by AICTE | Affiliated to OU | Accredited by NAAC & NBA

Department of Computer Science and Engineering

Project Title: Smart Life Scheduler.

Batch-ID:DS20

Student Team Members:

- 1) G.S Rohith, 2451-22-750-038
- 2) D Saathvik, 2451-22-750-041
- 3) N Pehlaj, 2451-22-750-044

Guide: T Laxmi, Asst Prof, Dept. of CSE, MVSREC

Problem Statement: In today's fast-paced environment, individuals struggle to balance demanding schedules with maintaining proper health. Traditional task scheduling tools help users manage daily activities but fail to consider their physical and mental well-being. Similarly, existing health-monitoring applications operate independently, offering no connection between a user's workload, habits, and health conditions. As a result, users lack awareness of how factors like sleep quality, stress levels, exercise patterns, and daily tasks influence one another. This leads to poor decision-making and unhealthy routines. Therefore, there is a need for an integrated system that unifies task planning, health tracking, data analysis, and automated health reporting to support balanced lifestyle management.

Project Scope:

1. Build a mobile app for task scheduling and health monitoring.
2. Integrate APIs like Google Fit/Apple Health for activity data.
3. Use Firebase/MongoDB for data management and ML for analysis.
4. Generate automated health reports with insights and trends.

Project Objectives:

1. Provide an easy tool to plan daily tasks.
2. Track, sleep, exercise, stress and symptoms.
3. Analyze links between tasks and health patterns.
4. Offer personalized health reports and suggestions.

Application Areas	Challenges
1. Personal Health Tracking-Helps users monitor sleep, stress and activity daily. 2. Task and Routine Management-Allows users to plan and organize their daily tasks efficiently. 3. Fitness Monitoring-Tracks steps, workouts and physical activity trends. 4. Employee Wellness Programs-Supports companies in improving employee's health and productivity. 5. Preventive Health-Provides early insights into unhealthy patterns through reports.	1. Data Integration Issues-Syncing accurate data from Google Fit/Apple Health can be complex. 2. Privacy and Security Concerns-Sensitive health data must be protected with strong security. 3. Limited ML Training Data-Health insights may be Less accurate with small datasets. 4. User Engagement-Users may not consistently input their tasks or symptoms. 5. Cross-platform Performance-Ensuring smooth functionality on both android and iOS is difficult.

Literature Survey

SL	Major Reference(s)	Finding(s)
1	Google Fit API Documentation	Provides accurate step count, activity type and heart-rate success for integrating real time health data into mobile apps.
2	Apple Health Kit API Documentation	Enables secure collection of sleep, heart rate and exercise metrics from IOS devices for personalized health monitoring.
3	Firebase Fire store Documentation	Offers scalable, real-time cloud storage for user tasks, health records and reports with built in authentication and security rules.
4	Research on Digital Wellness & Productivity Tools	Suggests that task scheduling apps reduce stress and increase productivity when combined with timely reminders and health insights.
5	WHO Health Monitoring Guidelines	Highlights the importance of tracking the daily physical activities of an individual for early detection of lifestyle-based health records.

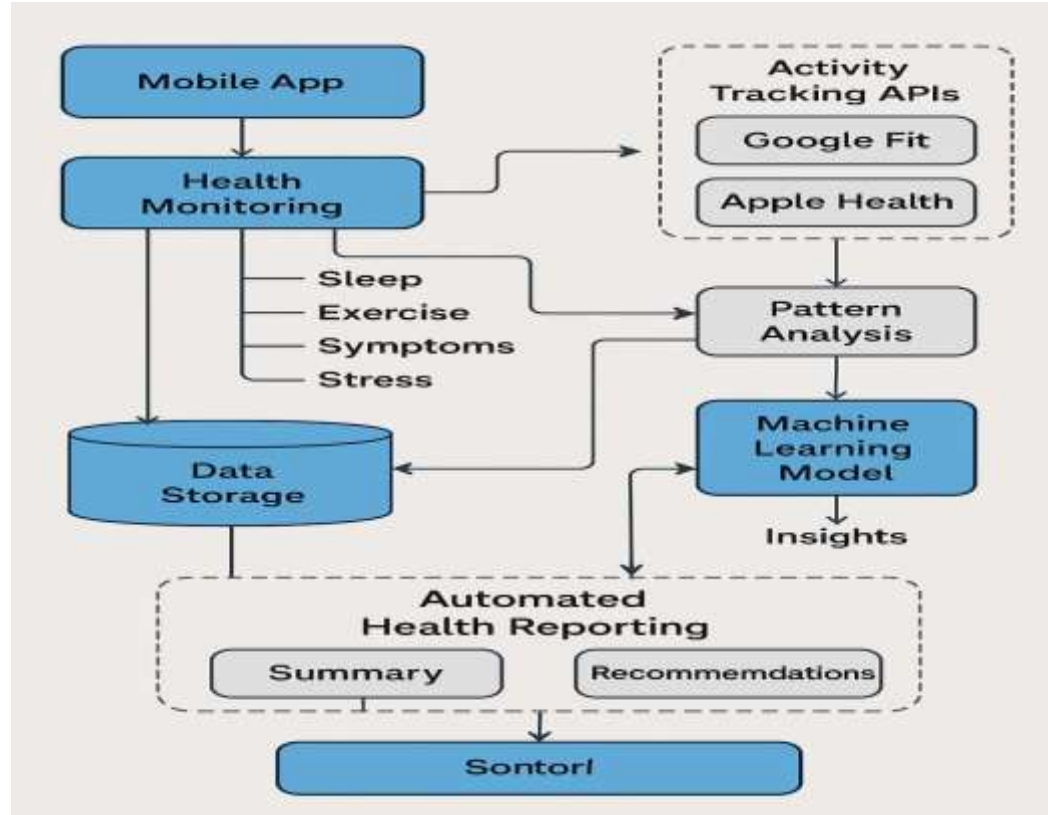
Comparison Table

SI no	Title	Paper/Reference	Methodology Used	Dataset Used
1.	Smart Life Scheduler: An AI-Based Personalized Daily Scheduling System.	Proposed System.	Machine Learning, Rule-Based Logic, AI Chat bot.	User schedules, personal preferences, lifestyle data.
2.	AI-Driven Task Scheduling and Productivity Optimization.	Related Work – 2022.	Machine Learning Algorithms.	User activity logs and task history.
3.	Health-Aware Lifestyle Recommendation System.	Related Work – 2021.	Data Analytics and Recommendation Models.	Health records and fitness tracking data.
4.	Automated Personal Planner Using Rule-Based Systems.	Related Work – 2020.	Rule-Based Expert System.	Static user schedule datasets.

Performance Metrics, Domain, Limitations & Future Scope Table:

Sl. No.	Performance Metrics	Domain	Limitations / Research Gaps	Future Scope
1.	Schedule accuracy, task completion rate	AI-Based Scheduling	Does not consider health parameters	Health-aware personalized scheduling
2.	User productivity improvement	Smart Lifestyle Management	Static schedules without adaptation	Dynamic rescheduling using AI
3.	Health balance, wellness score	Health Informatics	No integration with fitness apps	Integration with wearable devices
4.	User satisfaction, system adaptability	Intelligent Systems	Rule-based and non-learning systems	ML-driven adaptive life scheduling models

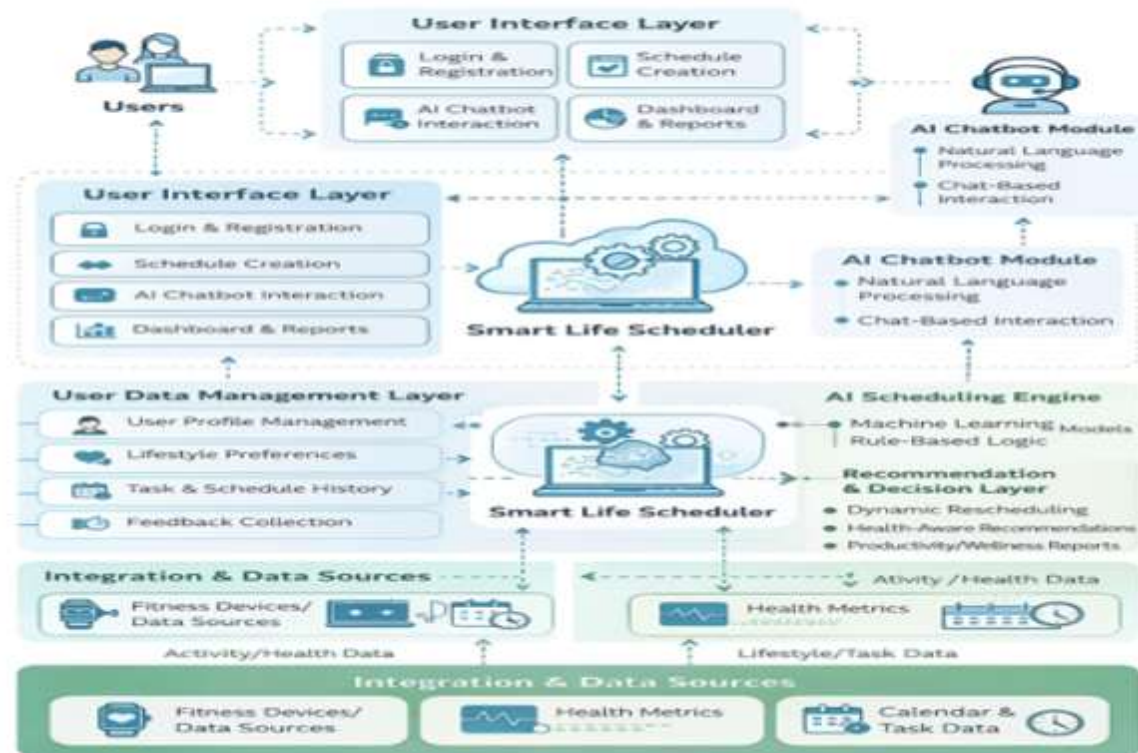
Generic Architecture of the Existing Systems:



Software / Hardware requirements for development / deployment

Tool / Technology	Usage
Flutter/React Native	Used to build a cross-platform mobile app with a smooth and responsive UI.
Node js/Django REST	Used to create backend API for data handling, Authentication and communication between app and database.
Firebase Fire store/MongoDB/SQL	Used to store tasks, health data, activity logs and system generated reports securely.
Python (Scikit- learn, Pandas, Numpy)	Used to perform health data analysis and run machine learning models for pattern detection and insights.
Google Fit API/Apple Health kit API	Used to fetch step count, sleep data, heart rate and physical activity directly from a user device.
Firebase Cloud Messaging/Report lab	Used for sending real time push notifications and generating downloadable Pdf health reports.

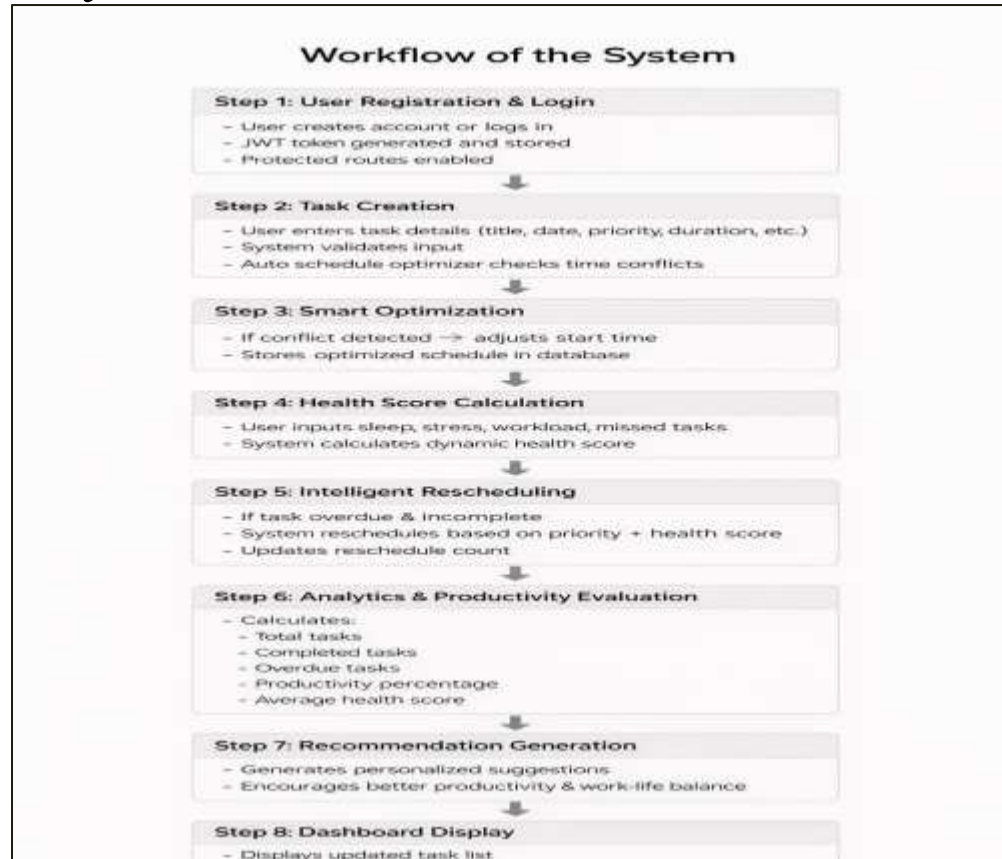
Proposed System Architecture Smart Life Scheduler



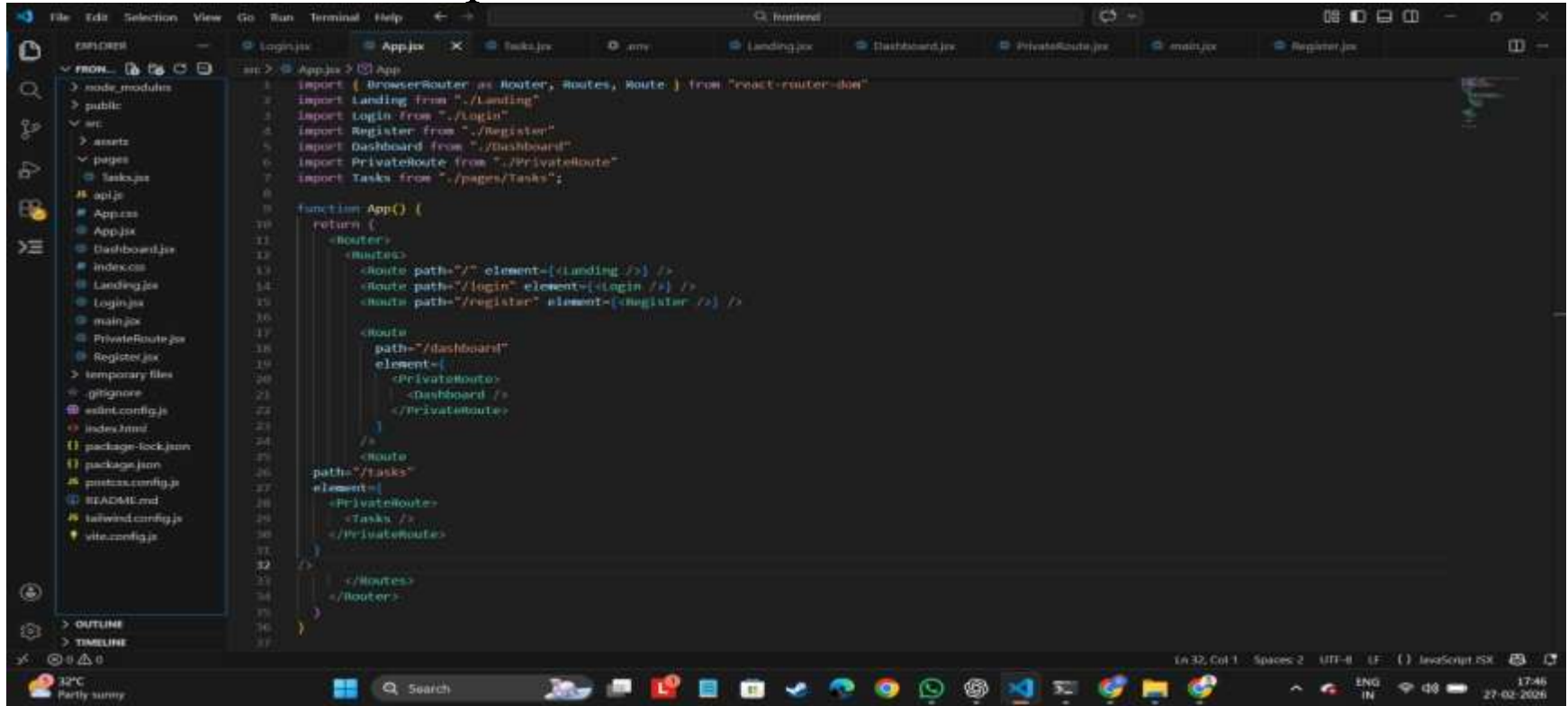
Module Description:

Name of Module	Description
User Authentication Module	Handles User Registration, Login, JWT-based authentication and secure session management using token verification and blacklist mechanism.
Task Management Module	Allows Users to create, view, update, toggle and delete tasks with full CRUD functionality.
Auto Scheduling Optimization Module	Detects scheduling conflicts between tasks. Automatically adjusts task start times to prevent overlapping and optimize time utilization.
Smart Rescheduling & Health Scoring Module	Analyzes user sleep hours, stress levels, workload, and missed tasks. Calculates a health score and intelligently reschedules overdue tasks based on user well-being.
Analytics & Productivity Module	Computes key productivity metrics such as Total tasks, Completed tasks, Overdue tasks, Average health score. Provides insights for better decision-making.
Recommendation Engine Module	Generates personalized productivity and health recommendations. Uses user performance data to suggest improvements and optimize daily routines.
Dashboard Visualization Module	Displays structured summaries of tasks and analytics.
Deployment & Cloud Integration Module	Deploys the backend application on Render. Integrates MongoDB Atlas for scalable cloud-based database management.

Workflow of the System:



Environmental Setup:

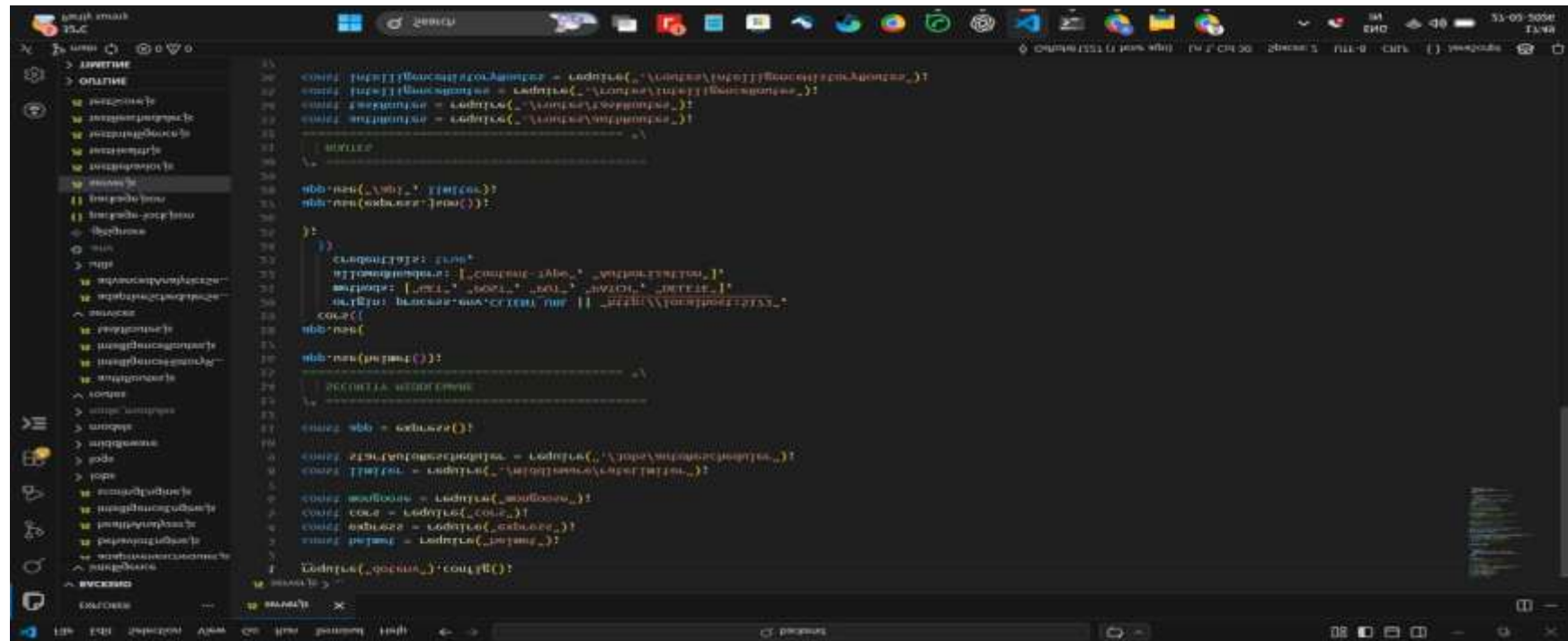


The Project is implemented using VS Code Text Editor.

Frontend:

A screenshot of a code editor with a dark theme. The left sidebar shows a file explorer with a project structure. The main editor area displays the code for 'Dashboard.jsx'. The code includes imports from 'react-router-dom', a function 'dashboard()' that uses 'useNavigate' and 'localStorage', and JSX for a sidebar and navigation buttons. The bottom status bar shows 'Ln 2, Col 1', 'Spaces: 2', 'UTF-8', 'CRLF', and 'JavaScript JSX'. The Windows taskbar is visible at the bottom with various icons and system information like '32°C Partly sunny' and '27-02-2026'.

```
File Edit Selection View Go Run Terminal Help
Dashboard.jsx
1 import { useNavigate } from "react-router-dom";
2
3 function dashboard() {
4   const navigate = useNavigate();
5
6   const handleLogout = () => {
7     localStorage.removeItem("token");
8     navigate("/login");
9   };
10
11   return (
12     <div className="flex h-screen bg-slate-950 text-white">
13
14       {/** Sidebar */}
15       <div className="w-64 bg-slate-900 border-r border-slate-800 flex flex-col">
16
17         <div className="p-6 text-2xl font-bold border-b border-slate-800">
18           Smart Life
19         </div>
20
21         <nav className="flex-1 p-4 space-y-2">
22
23           <button
24             onClick={() => navigate("/dashboard")}
25             className="w-full text-left px-4 py-2 rounded-lg hover:bg-slate-800 transition"
26           >
27             Overview
28           </button>
29
30           <button
31             onClick={() => navigate("/tasks")}
32             className="w-full text-left px-4 py-2 rounded-lg hover:bg-slate-800 transition"
33           >
34             Tasks
35           </button>
36
37           <button
```



Backend:

Backend running on Server:

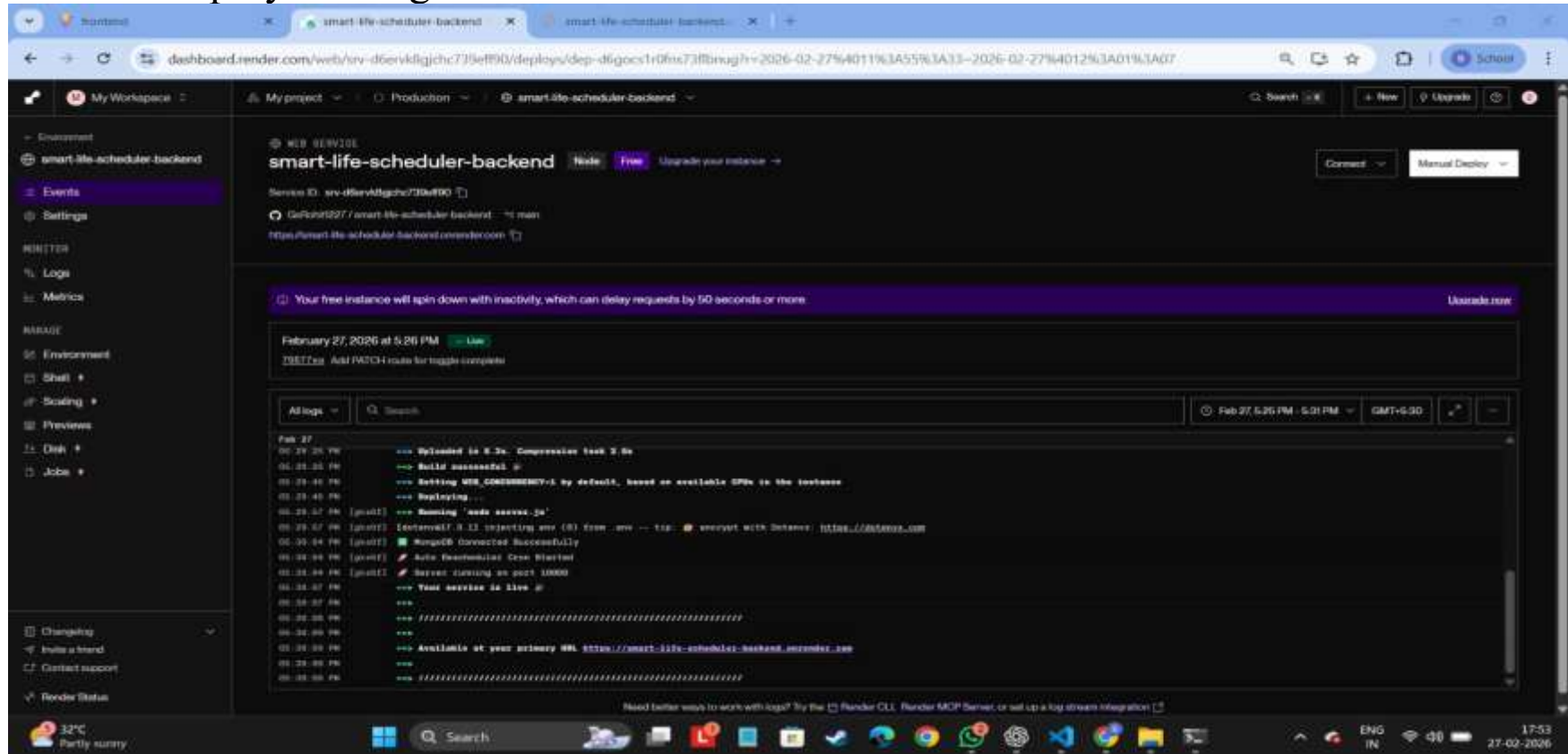
```
C:\WINDOWS\system32\cmd. x + -
Microsoft Windows [Version 10.0.26200.7922]
(c) Microsoft Corporation. All rights reserved.

C:\Users\dixit>cd Desktop
C:\Users\dixit\Desktop>cd smart-life-scheduler
C:\Users\dixit\Desktop\smart-life-scheduler>cd backend
C:\Users\dixit\Desktop\smart-life-scheduler\backend>npm run dev

> backend@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.11
[nodemon] to restart at any time, enter 'rs'
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node server.js'
[dotenv@17.3.1] injecting env (2) from .env -- tip: ⚙️ specify custom .env file path with { path: '/custom/path/.env' }
✔ MongoDB Connected Successfully
✔ Auto Rescheduler Cron Started
✔ Server running on port 5000
```

Backend deployed using render.com:



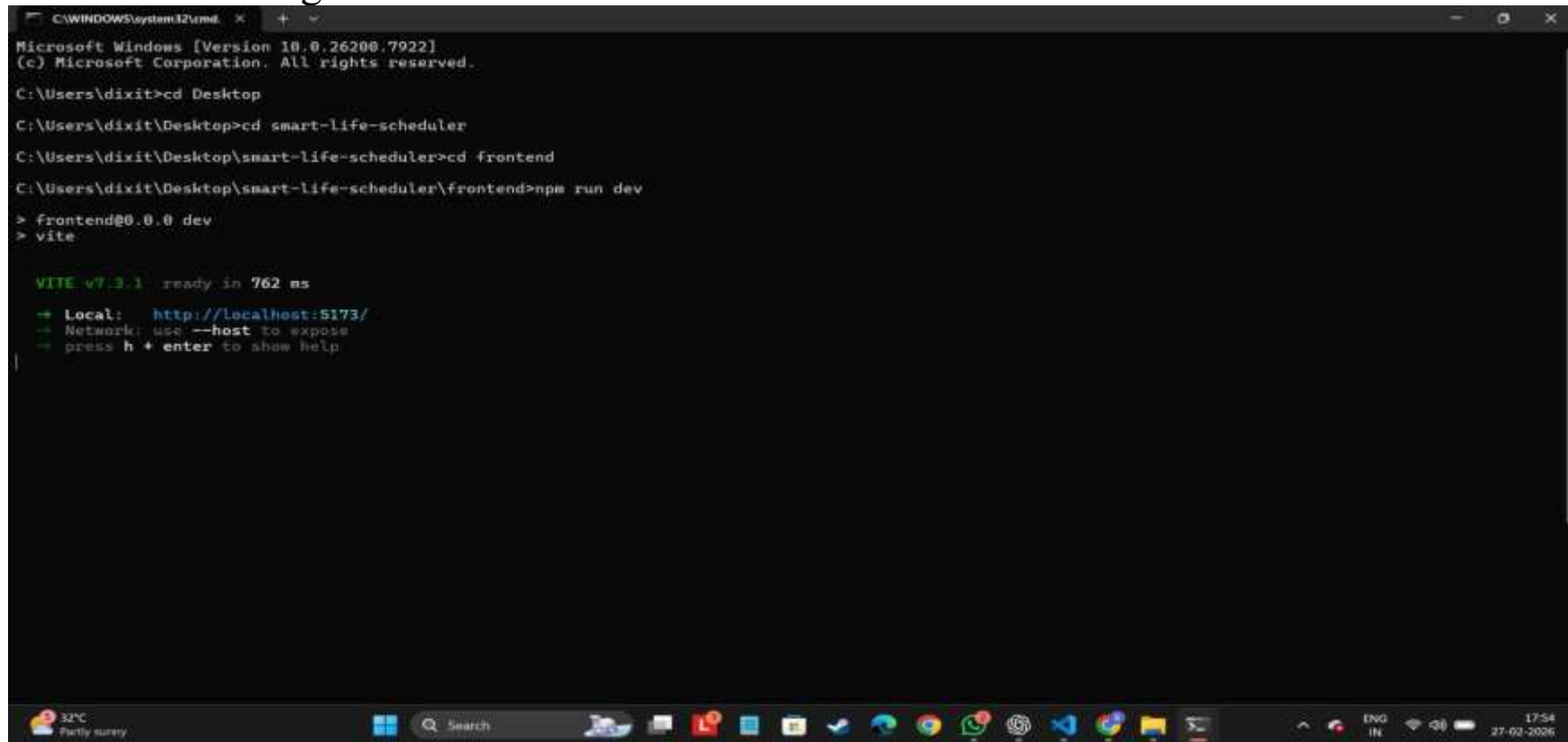
Frontend Running in local:

```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [Version 10.0.26200.7922]
(c) Microsoft Corporation. All rights reserved.

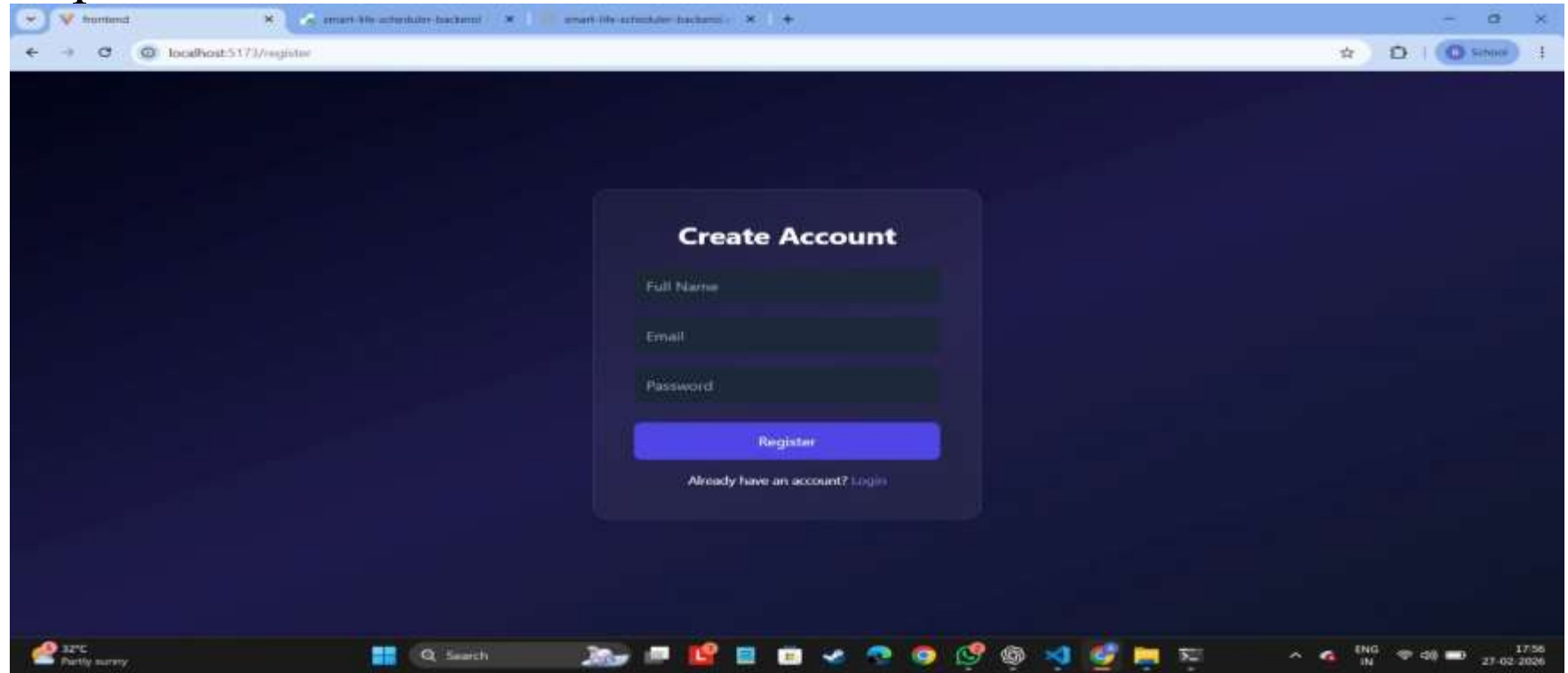
C:\Users\dixit>cd Desktop
C:\Users\dixit\Desktop>cd smart-life-scheduler
C:\Users\dixit\Desktop\smart-life-scheduler>cd frontend
C:\Users\dixit\Desktop\smart-life-scheduler\frontend>npm run dev

> frontend@0.0.0 dev
> vite

VITE v7.3.1 ready in 762 ms
➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
➔ press h + enter to show help
```

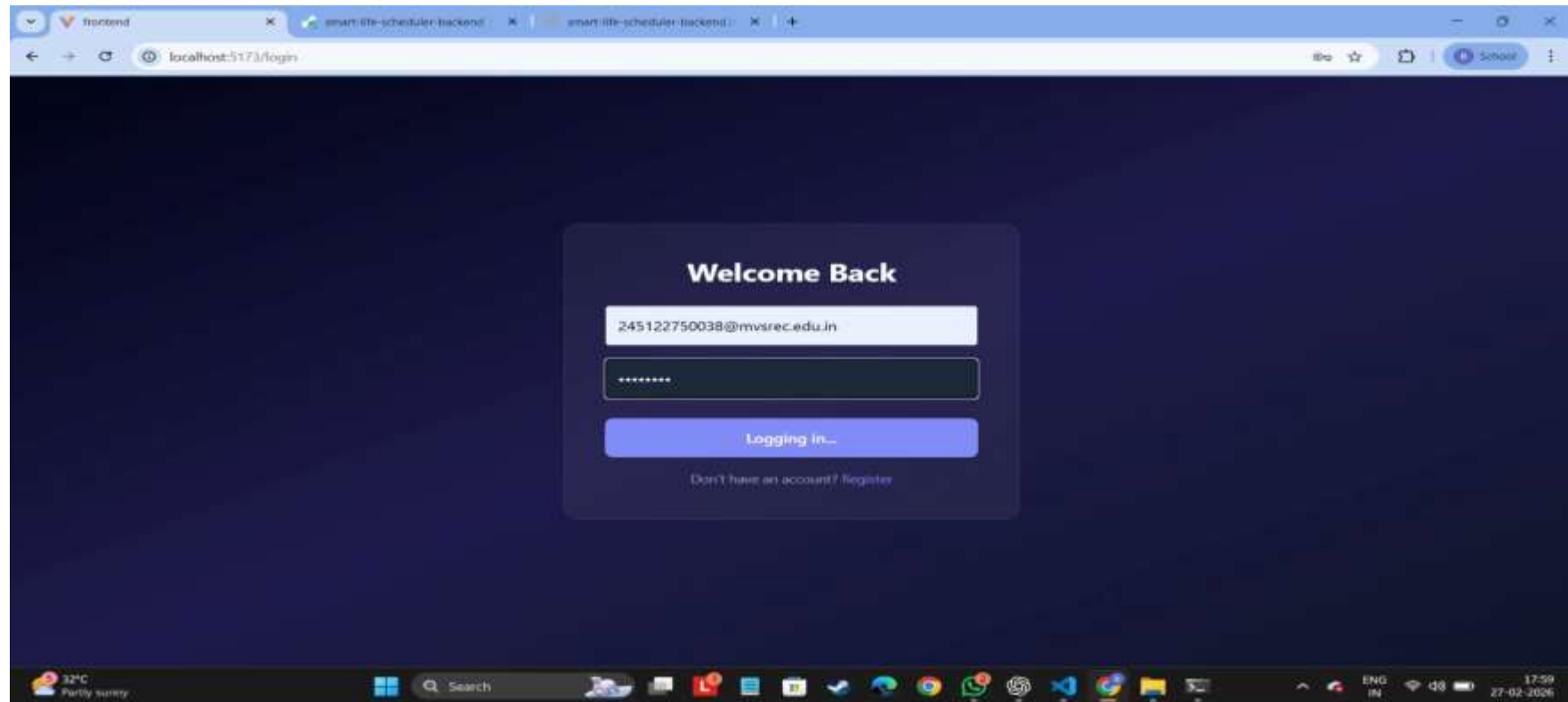


Implementation:

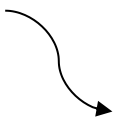


Register page:

Login page: 



Dashboard:



Smart Life

Dashboard Overview

Welcome Back 🌟

Overview

Tasks

Analytics

Reports

Settings

Logout

Total Tasks

0

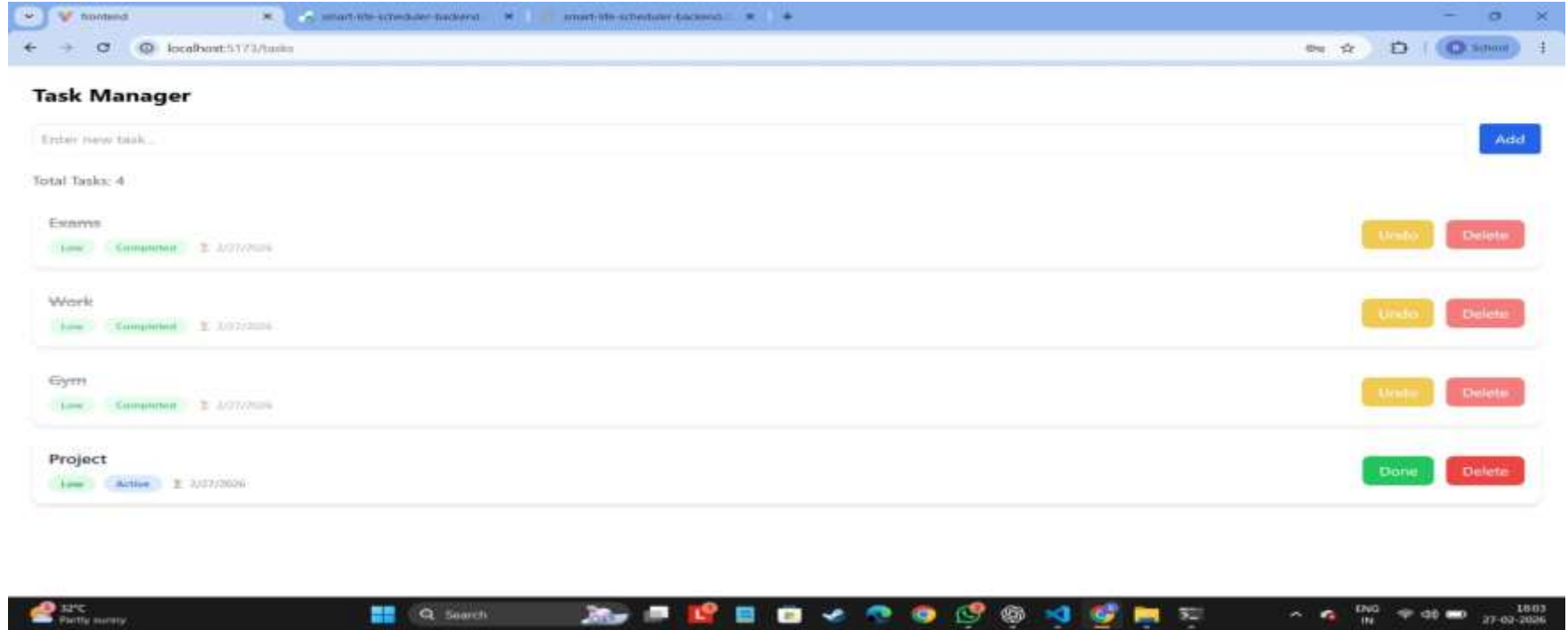
Completed

0

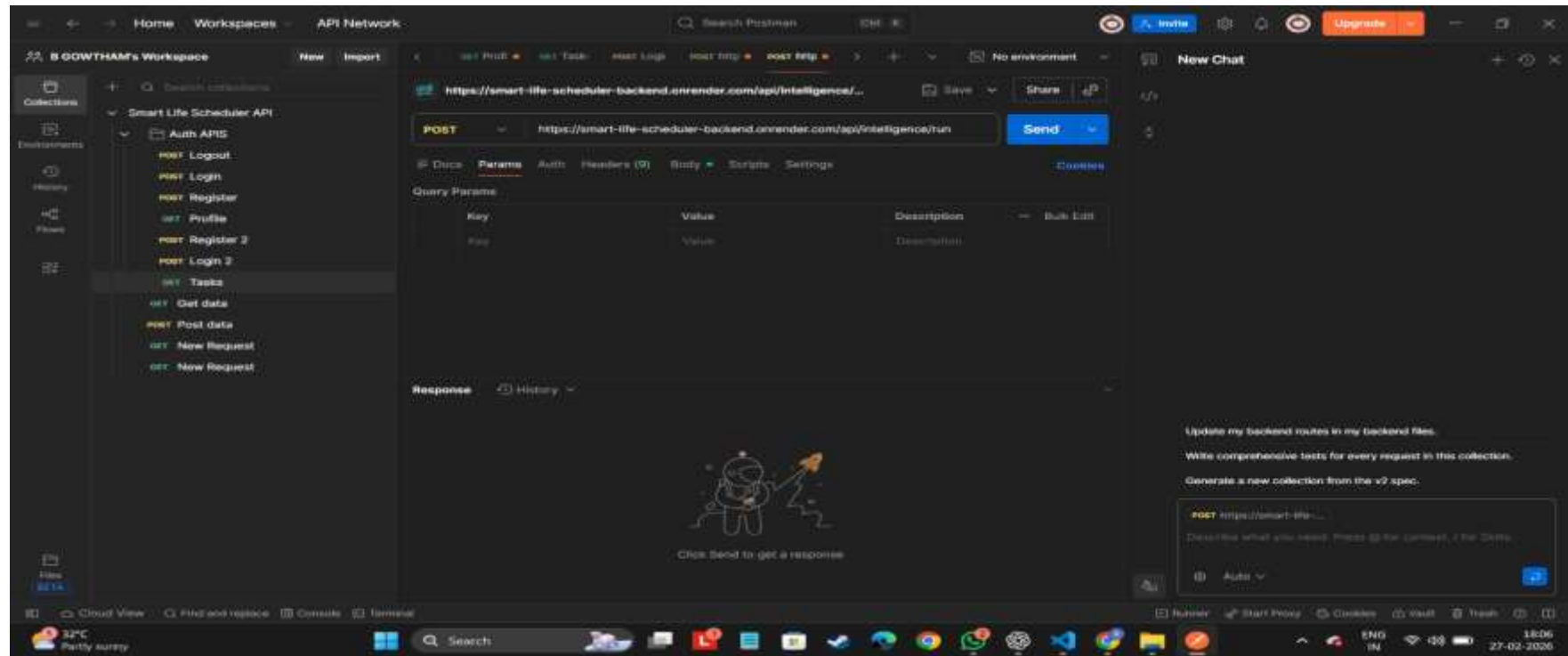
Productivity

0%

Windows taskbar: 27-02-2026 17:59



Task manager: 
The image reflects completed tasks and pending tasks.



Requests are done using the postman.com.

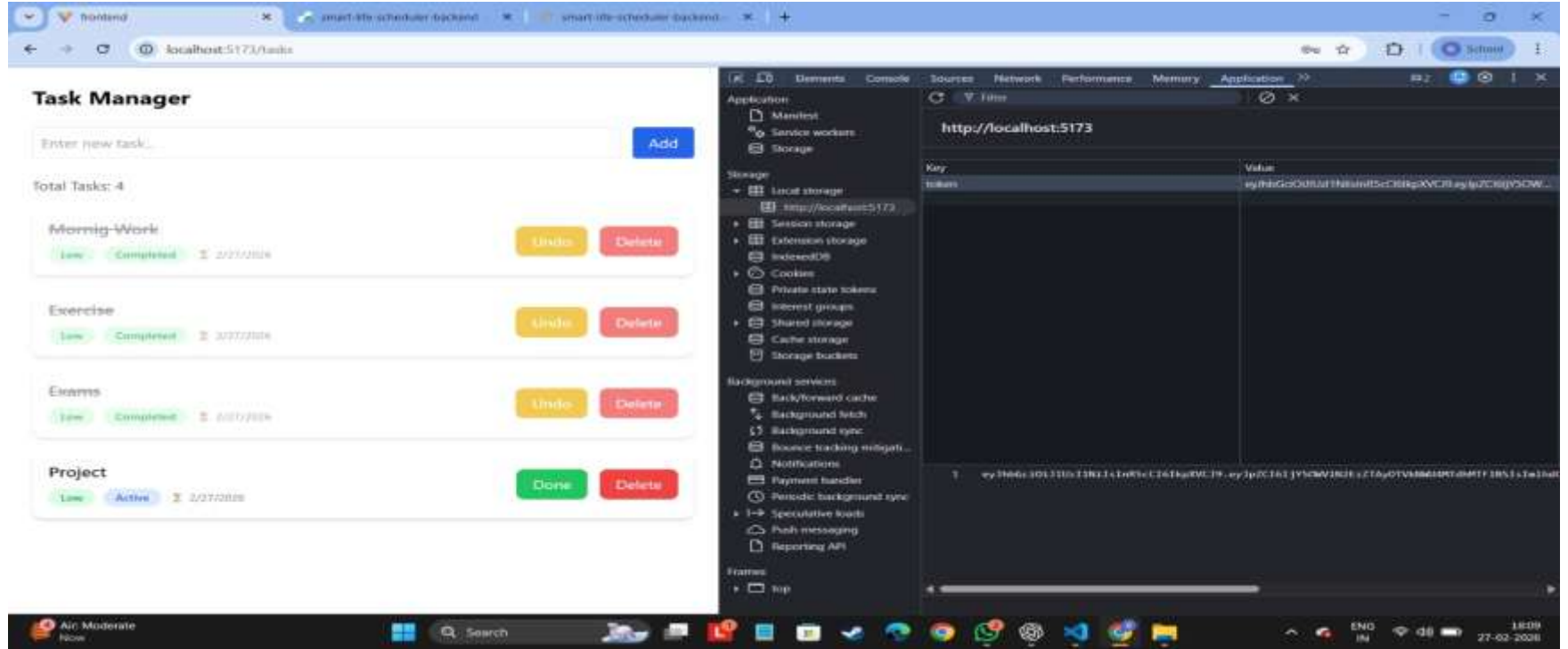
The screenshot displays a web browser window with the address bar showing `localhost:5173/tasks`. The main content area features a "Task Manager" interface with a form to "Enter new task..." and an "Add" button. Below the form, there are four task entries: "Morning Work", "Exercise", "Exams", and "Project". Each entry has a "Live" button, a "Completed" status, a date, and "Undo" and "Delete" buttons. The "Project" task has a "Done" button instead of "Undo".

The browser's developer tools are open, showing the "Network" tab. The "Fetch/XHR" filter is selected, and a list of network requests is displayed. The table below shows the details of these requests:

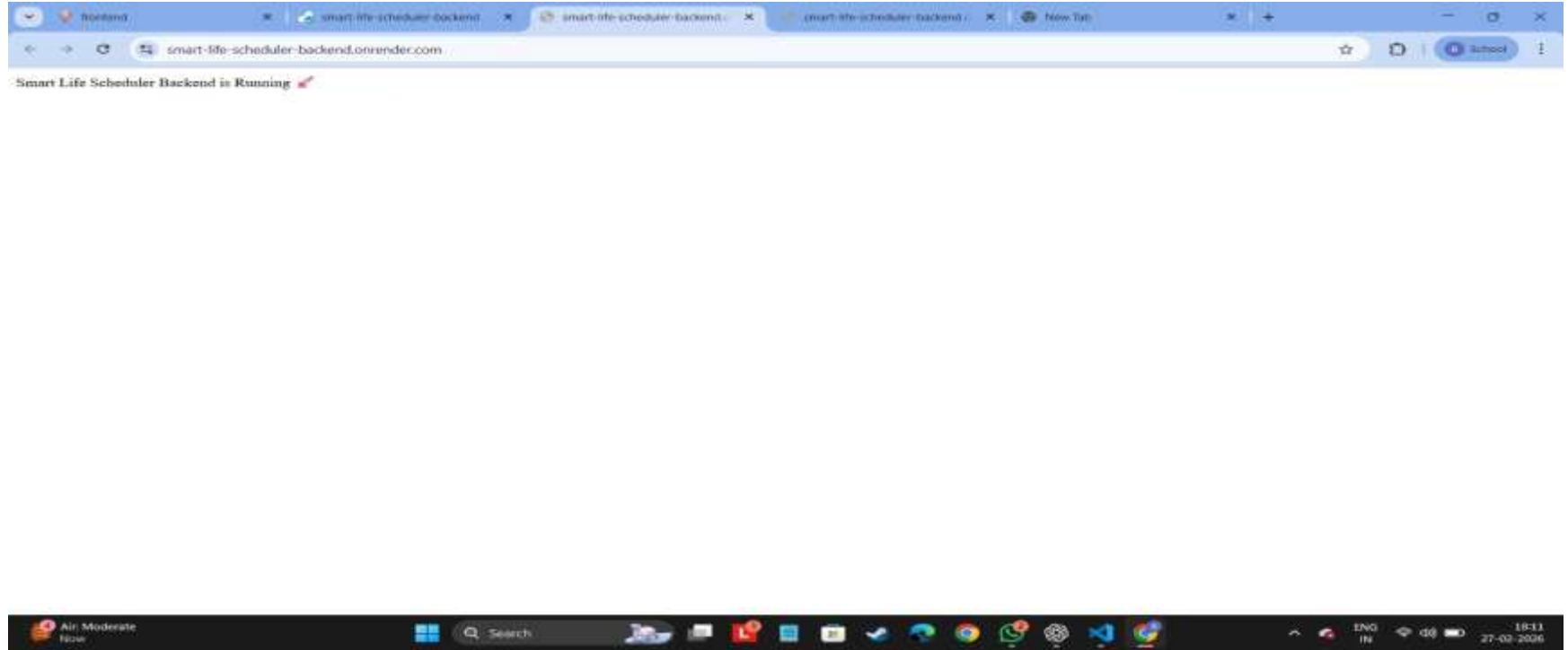
Name	Status	Type	Initiator	Size	Time
69a1902747f9625d0603144d	204	preflight	Preflight	0.0 kB	3.14 ms
69a1902747f9625d0603144d	200	xhr	tasks.jsx:54	1.0 kB	1.25 s
tasks	204	preflight	Preflight	0.0 kB	349 ms
tasks	200	xhr	tasks.jsx:10	1.2 kB	1.25 s
69a1902747f9625d0603144d	200	xhr	tasks.jsx:54	1.0 kB	1.24 s
tasks	200	xhr	tasks.jsx:10	1.2 kB	1.15 s
69a189c47f96d5d0603142b	204	preflight	Preflight	0.0 kB	295 ms
69a189c47f96d5d0603142b	200	xhr	tasks.jsx:54	1.0 kB	1.25 s
tasks	204	preflight	Preflight	0.0 kB	306 ms
tasks	200	xhr	tasks.jsx:10	1.2 kB	1.23 s
69a189c47f96d5d0603142b	200	xhr	tasks.jsx:54	1.0 kB	1.20 s
tasks	200	xhr	tasks.jsx:10	1.2 kB	1.25 s

The bottom of the browser window shows the Windows taskbar with the date and time set to 18:08 on 27-02-2026.

Network Status working properly in the dev tools of the browser.



Local storage is working and generating the JWT_SECRET token.



Deployed Backend link: <https://smart-life-scheduler-backend.onrender.com/>

Sustainability

The proposed Smart Daily Scheduler with Integrated Health Monitoring and Automated Reporting is highly sustainable across environmental, economic, and social dimensions. By replacing paper-based planners, manual health logs, and physical reports, the system significantly reduces paper consumption and minimizes the environmental footprint associated with traditional scheduling and health-tracking methods. The solution is developed using open-source and resource-efficient technologies such as Flutter/React Native, Firebase/MongoDB, and Python-based machine learning, making it cost-effective to build, deploy, and maintain over time. Its modular architecture—combining task management, health tracking, ML-driven insights, and automated reporting—ensures long-term technological sustainability and allows easy integration of new features or additional health metrics without major redesign.

Thank You!



Project Title: Smart life Scheduler

Batch-ID:DS20

Student Team Members:

- 1) G.S Rohith, 2451-22-750-038
- 2) D Saathvik, 2451-22-750-041
- 3) N Pehlaj, 2451-22-750-044

Guide: T Laxmi, Asst Prof, Dept. of CSE, MVSREC